

Software Design Document (SDD)

Módulo de Verificación de Nombres — search_name

Empresa/Dependencia: Entidades Financieras (CNBV / PLD-FT)

Tabla de Control de Versiones

Versión	Fecha	Autor(es)	Descripción del Cambio
1.0	2026-06-11	Equipo de Desarrollo	Versión inicial: motor de matching difuso, modelo NameRecord
2.0	2026-06-11	Equipo de Desarrollo	Incorpora modelos Oficio y PersonaCNBV, comando import_cnbv, vistas lista_oficios y comparar_oficio

Integrantes:

- Gerardo Maldonado Felix
- Sebastian Ortiz Lopez
- Angel David Carlos Silva
- Carlos Bernardo Gomez Garcia
- Luis Ruben Ramos Cortes
- Ruben Jimenez Navarro

1. Propósito del Diseño

Objetivo: Definir el diseño técnico del prototipo search_name, describiendo su estructura interna, sus componentes, sus decisiones de implementación y la forma en que el sistema resuelve tanto la búsqueda difusa manual de nombres como la comparación automatizada de oficios CNBV contra la base de clientes en el contexto de cumplimiento PLD-FT.

Alcance: Este diseño cubre el prototipo funcional construido sobre Django 5.x con tres modelos (NameRecord, Oficio, PersonaCNBV), un motor de matching textual y fonético, un

parser de XMLs CNBV, tres vistas web y un conjunto de datos ficticios para demostración. No cubre la integración final con datos reales ni la conexión operativa al sistema legado.

2. Arquitectura General

2.1 Estilo Arquitectónico

El sistema adopta el patrón nativo de Django: Modelo-Vista-Template (MVT). La separación de responsabilidades es la siguiente:

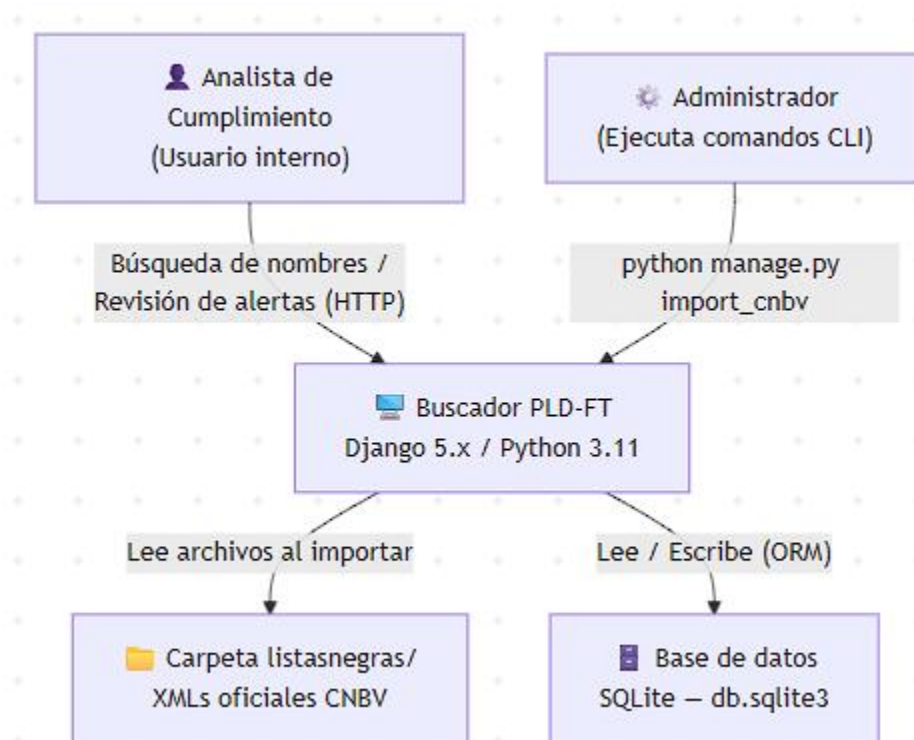
- Modelo (models.py): define la estructura de datos y la persistencia en SQLite.
- Vista (views.py): coordina la lógica; recibe la petición HTTP, invoca el motor de matching o el motor de comparación, y envía los resultados al template.
- Template (*.html): presenta los resultados al usuario en HTML.
- Módulo de matching (matching.py): lógica de comparación difusa, completamente desacoplada.
- Comando de importación (import_cnbv.py): parser XML y carga de oficios, desacoplado de la vista.

2.2 Diagrama de Contexto (texto)

El analista de cumplimiento accede desde un navegador web al sistema Django. Puede realizar dos flujos principales:

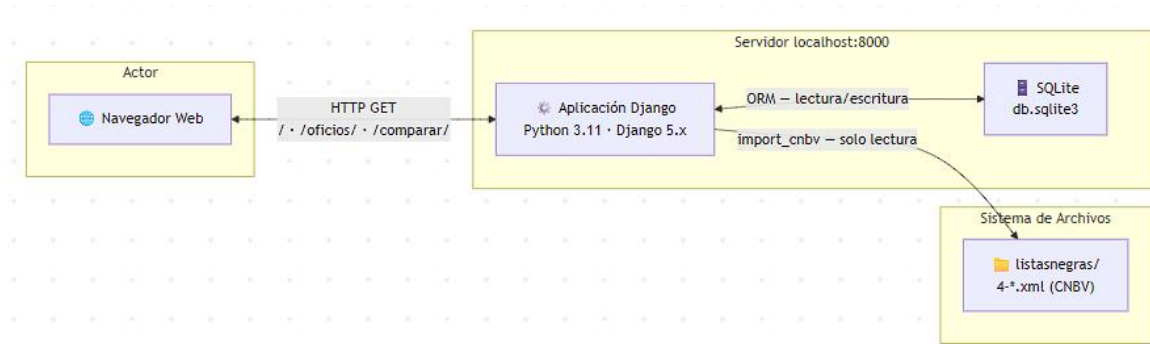
(A) Búsqueda manual: ingresa un nombre en / y el sistema compara contra NameRecord.

(B) Flujo CNBV: el administrador ejecuta 'python manage.py import_cnbv <carpeta>' para cargar los XMLs; luego el analista accede a /oficios/, selecciona un oficio y navega a /oficios/<folio>/comparar/ para ver los resultados automáticos. No existe comunicación con sistemas externos en el prototipo.



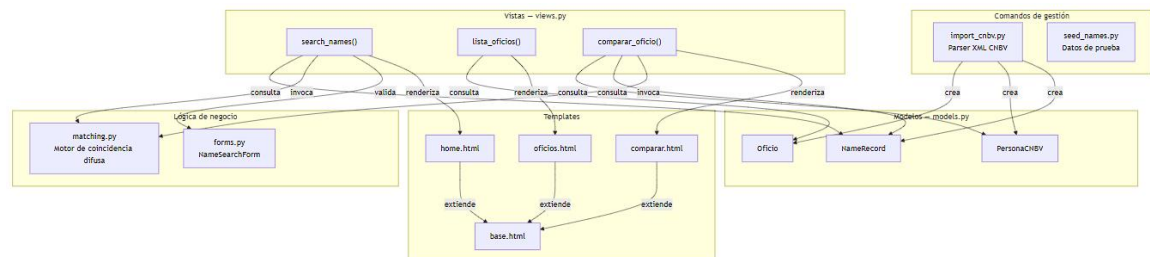
2.3 Diagrama de Contenedores (texto)

Contenedor	Responsabilidad	Tecnología
Navegador web	Cliente HTTP; no requiere JavaScript.	Cualquier navegador moderno
Aplicación Django (search_name)	Servidor web de desarrollo; expone /, /oficios/, /oficios/<folio>/comparar/ y /admin/.	Python 3.11 + Django 5.2.5
Base de datos SQLite	Archivo db.sqlite3 local; tablas coincidencias_namerecord, coincidencias_oficio, coincidencias_personacnbv.	SQLite 3
Carpeta listasnegras/	Repositorio local de archivos XML de oficios CNBV. Solo se lee durante el import.	Sistema de archivos



2.4 Rutas URL

URL	Vista / Módulo	Nombre	Descripción
/	coincidencias.views.search_names	search_names	Búsqueda difusa manual de nombres (GET).
/oficios/	coincidencias.views.lista_oficios	lista_oficios	Lista de oficios CNBV importados.
/oficios/<folio>/comparar/	coincidencias.views.comparar_oficio	comparar_oficio	Comparación de personas del oficio vs. base de clientes (GET, parámetro min_score).
/admin/	django.contrib.admin	admin:index	Panel de administración Django.



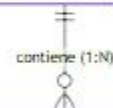
3. Componentes Principales

3.1 models.py — Tres modelos

El módulo define tres modelos persistidos en SQLite.

NameRecord			
int	id	PK	
varchar	nombre		
varchar	paterno		
varchar	materno		
varchar	rfc		
varchar	origen		
varchar	folio_ref		
varchar	search_full_name		INDEX
varchar	normalized_name		INDEX
datetime	created_at		
datetime	updated_at		

Oficio			
int	id	PK	
int	folio		UNIQUE INDEX
varchar	numero_oficio		
varchar	numero_expediente		
date	fecha_publicacion		
int	dias_plazo		
varchar	autoridad_nombre		
varchar	referencia		
bool	tiene_aseguramiento		
varchar	archivo_xml		
datetime	importado_en		



PersonaCNEV			
int	id	PK	
int	oficio_id	FK	
int	persona_id		
varchar	tipo_persona		
varchar	nombre		
varchar	paterno		
varchar	materno		
varchar	rfc		
varchar	search_full_name		INDEX
varchar	normalized_name		INDEX

Modelo NameRecord — base de datos de clientes de la entidad financiera:

Campo	Tipo	Descripción
nombre	CharField(120)	Nombre(s) de pila.
paterno	CharField(80)	Apellido paterno.
materno	CharField(80)	Apellido materno (puede vacío).
rfc	CharField(20)	RFC (opcional).
origen	CharField(40)	Fuente del registro.
folio_ref	CharField(50)	Referencia del oficio asociado.
search_full_name	CharField(180) INDEX	Nombre buscable derivado (auto).
normalized_name	CharField(180) INDEX	Nombre normalizado

		derivado (auto).
created_at / updated_at	DateTimeField	Timestamps de creación y modificación.

Modelo Oficio — encabezado de cada oficio CNBV:

Campo	Tipo	Descripción
folio	IntegerField UNIQUE INDEX	Número de folio del oficio (clave de unicidad).
numero_oficio	CharField(80)	Número oficial (ej. 214-4/27921410/2026).
numero_expediente	CharField(80)	Número de expediente interno CNBV.
solicitud_siara	CharField(60)	Clave SIARA (ej. UIFB/2026/001147).
anio	IntegerField	Año del oficio.
area_clave / area_descripcion	CharField	Clave y descripción del área CNBV.
fecha_publicacion	DateTimeField	Fecha en que se publicó el oficio.
dias_plazo	IntegerField	Días de plazo para responder (usualmente 1).
autoridad_nombre	CharField(220)	Nombre de la autoridad emisora (UIF).
referencia / referencia1	CharField(220)	Tipo de solicitud y clave de referencia.
tiene_aseguramiento	BooleanField	Si el oficio incluye aseguramiento.
archivo_xml	CharField(220)	Nombre del archivo XML de origen.
importado_en	DateTimeField(auto_now_add)	Timestamp de importación.

Modelo PersonaCNBV — cada persona listada en un oficio:

Campo	Tipo	Descripción
oficio	ForeignKey(Oficio)	Oficio al que pertenece esta persona.
persona_id	IntegerField	ID de persona dentro del oficio.
caracter	CharField(60)	Carácter (ej. Investigado).
tipo_persona	CharField(20)	Tipo: 'Física' o 'Moral'.
nombre	CharField(220)	Nombre(s) de pila o razón social (moral).
paterno	CharField(80)	Apellido paterno (vacío para morales).
materno	CharField(80)	Apellido materno (vacío para morales).
rfc	CharField(20)	RFC (puede vacío en SolicitudPartes, completo en PersonasSolicitud).
relacion	CharField(120)	Relación con el investigado

		principal.
domicilio	CharField(320)	Domicilio registrado.
complementarios	CharField(220)	Datos complementarios (ej. ACUERDO: 173/2026).
search_full_name	CharField(300) INDEX	Nombre buscable derivado (auto).
normalized_name	CharField(300) INDEX	Nombre normalizado derivado (auto).

Los tres modelos implementan `prepare_search_fields()` invocado automáticamente en `save()`, garantizando que `search_full_name` y `normalized_name` siempre estén actualizados.

3.2 matching.py — Motor de Coincidencia

Módulo central. Completamente desacoplado de Django; puede probarse en forma aislada.

Función	Responsabilidad
<code>clean_name_piece(value)</code>	Elimina espacios múltiples y caracteres de control.
<code>_dedupe_name_parts(parts)</code>	Elimina partes duplicadas de una lista comparando versiones normalizadas.
<code>build_search_name(nombre, paterno, materno)</code>	Construye el nombre buscable evitando repetir tokens ya presentes en otro campo.
<code>normalize_name(name)</code>	Convierte a minúsculas, elimina acentos (<code>unidecode</code>), elimina caracteres no alfanuméricos, colapsa espacios.
<code>tokens(name)</code>	Devuelve la lista de tokens del nombre normalizado.
<code>phonetic_token(token)</code>	Aplica Metaphone a un token; devuelve su código fonético.
<code>_token_overlap_score(tokens_a, tokens_b)</code>	Porcentaje de tokens comunes entre dos listas (0-100).
<code>_phonetic_scores(query, candidate)</code>	Calcula <code>phonetic_token_sort</code> , <code>phonetic_overlap</code> y su combinación ponderada (70/30).
<code>score_name(query, candidate)</code>	Calcula todas las métricas y devuelve un dict con 'overall' y métricas individuales.
<code>explain_score(metrics)</code>	Devuelve texto explicando el motivo principal de la coincidencia.
<code>find_name_matches(query, candidates, min_score, limit)</code>	Orquesta la búsqueda: itera candidatos, calcula score, filtra por umbral, ordena y limita.

3.3 import_cnbv.py — Comando de Importación XML

Comando Django (`python manage.py import_cnbv <carpeta>`) que implementa el flujo completo de importación de oficios CNBV.

- Acepta la ruta de la carpeta raíz como argumento posicional.
- La bandera `--reset` elimina todos los oficios existentes antes de importar.
- Escanea recursivamente buscando archivos que coincidan con el patrón `4-*.xml`.

- Parsea cada XML con `xml.etree.ElementTree` usando el namespace `http://www.cnbv.gob.mx`.
- Función auxiliar `_text(element, tag_name)`: extrae el texto de un subelemento con fallback.
- Crea un registro `Oficio` con todos los campos del encabezado.
- Itera `SolicitudEspecifica/PersonasSolicitud` y crea un `PersonaCNBV` por cada elemento.
- Omite folios ya existentes en la BD (unicidad por folio).
- Reporta en consola: folio, fecha, cantidad de personas y resumen final (importados/omitidos/errores).

3.4 views.py — Tres Vistas

search_names (GET /)

- Instancia `NameSearchForm` con los parámetros GET.
- Obtiene `query`, `min_score` y `limit` mediante `cleaned_or_default()`.
- Consulta `NameRecord` con `.values()` y llama a `find_name_matches()`.
- Renderiza `home.html` con resultados y total de registros.

lista_oficios (GET /oficios/)

- Consulta todos los `Oficio` con `prefetch_related('personas')` ordenados por fecha y folio.
- Renderiza `oficios.html` con la lista y el total de oficios.

comparar_oficio (GET /oficios/<folio>/comparar/)

- Obtiene el `Oficio` por folio (404 si no existe).
- Lee el parámetro `min_score` del GET (default 45, rango 0-100).
- Consulta todos los `NameRecord` con `.values()`.
- Por cada `PersonaCNBV` del oficio: llama a `find_name_matches(persona.search_full_name, clientes, min_score, limit=10)`.
- Construye una lista de resultados con `persona`, `matches` y bandera `tiene_coincidencias`.
- Cuenta `total_alertas` (personas con al menos un match).
- Renderiza `comparar.html` con el resumen y los resultados por persona.

3.5 forms.py — NameSearchForm

Formulario Django (GET) con tres campos:

- `query` (`CharField`, `max_length=180`, opcional): nombre a buscar.
- `min_score` (`IntegerField`, 0-100, `default=45`): umbral mínimo de coincidencia.
- `limit` (`IntegerField`, 1-100, `default=20`): número máximo de resultados.

3.6 Templates

Template	Vista que lo usa	Contenido principal
home.html	search_names	Formulario de búsqueda + tabla de resultados con

		porcentaje, motivo y métricas. Enlace a /oficios/.
oficios.html	lista_oficios	Tabla de oficios con folio, fecha, referencia, plazo (indicador color), conteo de personas y botón Comparar. Instrucción de carga si no hay oficios.
comparar.html	comparar_oficio	Encabezado del oficio + resumen (total personas, total clientes, total alertas). Por cada PersonaCNBV: tipo badge, nombre, RFC, carácter; tabla de clientes coincidentes con semáforo de color (rojo $\geq 80\%$, naranja $\geq 60\%$, amarillo $< 60\%$). Control de umbral ajustable.

3.7 admin.py — Tres AdminModels

AdminModel	list_display	search_fields	list_filter
NameRecordAdmin	full_name, normalized_name, created_at	full_name, normalized_name	created_at
OficioAdmin	folio, numero_oficio, fecha_publicacion, referencia, dias_plazo, importado_en	folio, numero_oficio, numero_expediente, referencia	fecha_publicacion, area_descripcion, tiene_aseguramiento
PersonaCNBVAdmin	search_full_name, tipo_persona, rfc, caracter, oficio	search_full_name, normalized_name, rfc	tipo_persona, caracter, oficio_fecha_publicacion

3.8 seed_names.py — Datos de Demostración

Comando (python manage.py seed_names) que inserta 24 registros ficticios en NameRecord para demostrar los escenarios de matching. Incluye: pares con errores tipográficos (Juan/Johan, Perez/Peres), variantes fonéticas (Cristian/Christian, Hernandez/Hernandes), abreviaciones (Luis Eduardo/Luis Eduard) y casos con nombre en un solo campo vs. separado. La bandera --reset elimina todos los registros antes de insertar.

4. Diseño de Base de Datos

4.1 Tablas

coincidencias_namerecord (base de clientes):

Columna	Tipo / Restricción	Descripción
id	INTEGER PK AUTOINCREMENT	Identificador único.
nombre	VARCHAR(120)	Nombre(s) de pila.
paterno	VARCHAR(80)	Apellido paterno.
materno	VARCHAR(80)	Apellido materno (puede vacío).
rfc	VARCHAR(20)	RFC.
origen	VARCHAR(40)	Fuente del registro.
folio_ref	VARCHAR(50)	Referencia del oficio.
search_full_name	VARCHAR(180) INDEX	Nombre buscable (campo derivado).
normalized_name	VARCHAR(180) INDEX	Nombre normalizado (campo derivado).
created_at / updated_at	DATETIME	Timestamps.

coincidencias_oficio (oficios CNBV):

Columna	Tipo / Restricción	Descripción
id	INTEGER PK AUTOINCREMENT	Identificador único.
folio	INTEGER UNIQUE INDEX	Folio del oficio (clave de negocio).
numero_oficio	VARCHAR(80)	Número oficial.
numero_expediente	VARCHAR(80)	Número de expediente.
solicitud_siara	VARCHAR(60)	Clave SIARA.
anio	INTEGER	Año del oficio.
area_clave / area_descripcion	VARCHAR	Área CNBV.
fecha_publicacion	DATE	Fecha del oficio.
dias_plazo	INTEGER	Días de plazo de respuesta.
autoridad_nombre	VARCHAR(220)	Nombre de la autoridad.
referencia / referencia1	VARCHAR(220)	Tipo y clave de referencia.
tiene_aseguramiento	BOOLEAN	Si hay aseguramiento.
archivo_xml	VARCHAR(220)	Nombre del XML de origen.
importado_en	DATETIME	Timestamp de importación.

coincidencias_personacnbv (personas por oficio):

Columna	Tipo / Restricción	Descripción
id	INTEGER PK AUTOINCREMENT	Identificador único.
oficio_id	INTEGER FK → coincidencias_oficio	Oficio al que pertenece.
persona_id	INTEGER	ID de persona dentro del oficio.
caracter	VARCHAR(60)	Carácter (Investigado, etc.).
tipo_persona	VARCHAR(20)	Física o Moral.
nombre / paterno / materno	VARCHAR	Datos de identidad.

rfc	VARCHAR(20)	RFC.
relacion / domicilio / complementarios	VARCHAR	Datos adicionales del XML.
search_full_name	VARCHAR(300) INDEX	Nombre buscable (auto).
normalized_name	VARCHAR(300) INDEX	Nombre normalizado (auto).

4.2 Razón del Diseño con Modelos Separados

Se decidió separar Oficio y PersonaCNBV en lugar de una tabla única porque:

- Un oficio puede contener múltiples personas (relación 1:N).
- Los datos del encabezado del oficio son estables y no deben repetirse por persona.
- La clave de unicidad del oficio (folio) es independiente de las personas.
- Este diseño replica la estructura lógica del XML original de la CNBV.
- El modelo PersonaCNBV reutiliza el mismo mecanismo de campos derivados (search_full_name, normalized_name) que NameRecord, garantizando comparación homogénea.

5. Flujo de la Aplicación

Flujo A — Búsqueda difusa manual:

Paso	Descripción
1	El analista ingresa un nombre en / y ajusta umbral y límite.
2	HTTP GET → / con parámetros query, min_score, limit.
3	search_names() valida el formulario y consulta NameRecord.objects.values().
4	find_name_matches() calcula el score de cada candidato.
5	Se filtran por umbral, se ordenan y se limitan.
6	home.html renderiza la tabla de resultados.

Flujo B — Comparación de oficio CNBV:

Paso	Actor	Descripción
1	Admin	Ejecuta: python manage.py import_cnbv <carpeta_listasnegras>
2	import_cnbv	Escanea 4-*.xml recursivamente, parsea namespace CNBV, crea Oficio + PersonaCNBV por cada archivo nuevo.
3	Analista	Navega a /oficios/ — ve la lista de oficios con folios, fechas y conteo de personas.
4	Analista	Hace clic en 'Comparar' para el oficio de interés → /oficios/<folio>/comparar/
5	comparar_oficio()	Obtiene todas las PersonaCNBV del oficio y

		todos los NameRecord como candidatos.
6	comparar_oficio()	Por cada PersonaCNBV: llama a find_name_matches(persona.search_full_name, clientes, min_score, 10).
7	comparar_oficio()	Construye lista de resultados con tiene_coincidencias y cuenta total_alertas.
8	comparar.html	Muestra resumen (personas, clientes, alertas), tarjetas por persona con semáforo de color y tabla de clientes coincidentes.
9	Analista	Ajusta el umbral en la misma página y recalcula si es necesario.

6. Diseño del Algoritmo de Matching

6.1 Preparación del Nombre (build_search_name)

- Se limpian espacios múltiples de cada pieza (clean_name_pieza).
- Se unen nombre, paterno y materno cuando están disponibles.
- Si los tokens de paterno o materno ya están en nombre (campo completo), se omiten para no duplicar.
- Se eliminan partes duplicadas comparando sus versiones normalizadas (_dedupe_name_parts).

6.2 Normalización (normalize_name)

- unicode(): convierte Unicode a ASCII (elimina acentos, ñ → n).
- lower(): convierte a minúsculas.
- NON_ALNUM_RE.sub(): elimina todo lo que no sea letra, dígito o espacio.
- SPACE_RE.sub(): colapsa múltiples espacios en uno.

6.3 Métricas Textuales (rapidfuzz) y sus Pesos

Métrica	Peso	Justificación
token_sort_ratio	0.23	Ordena tokens antes de comparar. Mayor peso: resuelve el problema de orden de apellidos, el caso más frecuente en PLD-FT.
ratio	0.15	Similitud carácter a carácter. Sensible al orden y la posición.
token_set_ratio	0.15	Usa la intersección de tokens. Robusto ante nombres con partes extra o faltantes.
partial_ratio	0.15	Busca la subsecuencia más similar. Útil para nombres incompletos o abreviados.

Levenshtein normalizado	0.12	Número de ediciones mínimas normalizado. Complementa ratio.
Jaro-Winkler	0.10	Prioriza coincidencias al inicio de cadena. Útil para nombres propios con prefijo estable.
Fonética (Metaphone)	0.10	token_sort_ratio de códigos fonéticos (0.70) + overlap (0.30). Detecta variantes como HERNANDEZ/HERNANDES.

Suma total de pesos: 1.00

6.4 Ajustes del Ranking

- Penalización por longitud: si la proporción de longitudes es <0.45 y $\text{partial_ratio} > 90$, se aplica penalización para evitar que un string muy corto infle el score por ser subcadena.
- Coincidencia exacta tras normalización $\rightarrow \text{score} = 100.0$
- Mismos tokens en diferente orden $\rightarrow \text{score} = \max(\text{score}, 97.0)$

7. Decisiones Clave de Diseño

ID	Decisión	Justificación
DC-01	Separar matching.py de models.py y views.py	Permite probar el motor de forma aislada. Una versión futura puede reemplazar rapidfuzz por un modelo de ML sin tocar la vista.
DC-02	Campos separados en NameRecord y PersonaCNBV (nombre/paterno/materno)	Los oficios CNBV llegan con nombre partido entre campos. Separar permite construir variantes de candidato para maximizar la detección.
DC-03	Múltiples métricas ponderadas	Ninguna métrica aislada cubre todos los casos. La combinación hace el sistema más robusto frente a errores de captura, orden y fonética.
DC-04	token_sort_ratio con mayor peso (0.23)	El caso más frecuente en PLD-FT es el cambio de orden (apellido antes que nombre). Darle el mayor peso favorece la detección del escenario de mayor riesgo regulatorio.
DC-05	Variantes de candidato	Un cliente puede estar

	(<code>_candidate_variants</code>)	registrado con o sin apellido materno. Evaluar ambas variantes garantiza que no se pierda una coincidencia real.
DC-06	Parseo XML con ElementTree + namespace explícito	La CNBV usa el namespace <code>http://www.cnbv.gob.mx</code> . Usar ElementTree nativo evita dependencias extra; el <code>helper_tag()</code> centraliza el prefijo de namespace.
DC-07	Modelo Oficio separado de PersonaCNBV	Replica la estructura 1:N del XML original (un oficio, múltiples personas). Facilita mostrar el encabezado del oficio independientemente de las personas.
DC-08	SQLite para el prototipo	Sin dependencias externas; facilita la portabilidad y la demostración. En producción se migrará a PostgreSQL o MySQL.
DC-09	Umbral por defecto de 45%	Valor empírico que equilibra falsos positivos y negativos en los datos de demostración. Debe recalibrarse con datos reales.

8. Supuestos

- Los archivos XML de los oficios CNBV siguen el namespace `http://www.cnbv.gob.mx` con la estructura Expediente > SolicitudEspecificas > PersonasSolicitud.
- Los nombres pueden llegar completos o fragmentados entre los campos del registro.
- El apellido materno puede faltar en la consulta del analista o en el registro de la BD.
- Una coincidencia alta no confirma por sí sola identidad; es un insumo para la decisión del analista.
- Los pesos del algoritmo son preliminares; requieren validación con datos reales.

9. Limitaciones Conocidas

- Sin descarga automática: el sistema asume que los XML ya están disponibles localmente. La descarga desde el portal CNBV no está implementada.
- Sin conexión con datos reales de clientes.
- Sin autenticación: en el prototipo cualquiera con acceso puede consultar. En producción se requiere gestión de sesiones.

- Sin registro de auditoría: las consultas y comparaciones no se persisten. Necesario en producción para trazabilidad regulatoria.
- Escalabilidad: el algoritmo itera todos los registros en memoria. Para bases grandes se requiere una etapa previa de pre-filtrado (índice de trigramas o búsqueda vectorial).
- Los pesos del algoritmo pueden refinarse con muestras reales de la entidad financiera.